

Analysis of Consistent Hashing Strategies: Baseline vs. Proposed Adaptive Methods

Leonard Richertz (lr106)
Raha Gharehdaghi (rg152)

January 13, 2026

Reproducing Figures

To generate all figures used in this report, follow these steps:

1. Clone the project repository: `https://github.com/leonardrichertz/Probabilistic_Algorithms_and_Data_Structures_final_project`
2. Create and activate a Python virtual environment.
3. Install dependencies using: `pip install -r requirements.txt`
4. Run the simulation script: `python main_simulation.py`

1 Introduction

Consistent hashing distributes data across servers by mapping both servers and keys onto a logical ring, ensuring that adding or removing a server causes minimal data movement. However, its reliance on random placement leads to load imbalance: some servers are responsible for disproportionately large arcs of the ring.

To mitigate overload, systems often extend consistent hashing with bounded loads, enforcing a maximum capacity per server. However, traditional bounded-load handling relies on linear probing. This mechanism introduces the well-known cascading overflow problem—a single full server spills its load into its neighbor, which in turn becomes overloaded, causing a chain reaction that saturates an entire segment of the ring.

This report evaluates consistent hashing strategies designed to combat imbalance and overflow. These include two widely studied baselines—Random Jump Consistent Hashing and Power-of- k Choices as well as two new approaches, Threshold Power-of- k and Adaptive- k Consistent Hashing, which aim to further improve load distribution and system stability [1, 2].

2 Baseline Methods

First introduced over two decades ago, consistent hashing maps servers and keys to a unit circle using a hash function. Each key is assigned to the first server encountered when moving clockwise around the ring. While this design supports scalability and low migration costs, its reliance on purely random placement often leads to uneven load distribution, where some servers control disproportionately large arcs.

To address server overload, Google introduced Consistent Hashing with Bounded Loads, which caps each server’s capacity. When an object hashes to a full server, the system forwards the request to the next server on the ring. Although this prevents any single server from becoming overwhelmed, the linear-probing overflow strategy creates cascading overflows: once one server fills up, its excess traffic sequentially overloads its neighbors [3].

Rice University research such as “Revisiting Consistent Hashing with Bounded Loads” addresses this overflow issue by replacing linear probing with a cycle-free random jump mechanism. An initial hash maps the object to a location on the ring. If the target server is full, the algorithm “jumps” to a new random server index until a non-full bin is found. Instead of shifting to the next server, the algorithm rehashes the key using an incrementing counter ($\text{hash}(\text{key} + j)$), generating a deterministic but distinct candidate location on each overflow attempt. This rehashing avoids cycles created by naive recursive hashing. Objects are therefore redistributed across the cluster in a scattered fashion, preventing cascades and reducing hotspot formation [1].

Also at Rice, formalized in the patent “Consistent Hashing Using the Power of K Choices in Server Placement,” proposes improvements to both server placement and key assignment. Instead of placing new servers at random positions on the ring, the system evaluates k candidate positions and selects the optimal server. This dramatically reduces the initial imbalance created by random server placement ($k = 1$) [2].

Algorithm 1 Power-of- k Placement

```

1: Initialize empty list  $C$ 
2:  $j \leftarrow 0$ 
3: while  $|C| < k$  do
4:    $s \leftarrow \text{hash}(x + j)$ 
5:   if  $s$  is not full and  $s \notin C$  then
6:     add  $s$  to  $C$ 
7:   end if
8:    $j \leftarrow j + 1$ 
9: end while
10: Choose  $s^*$  in  $C$  with the smallest load
11: Place object  $x$  on  $s^*$ 

```

3 Proposed Methods

Best of both worlds: we propose combining the idea of optimal servers from the power of k choices, yet not keeping k fixed, resulting in more flexibility while maintaining the optimal server, having a threshold bound, and introducing randomness for cases where the optimal server is not good enough to increase distribution fairness, especially in skewed data.

3.1 ThresholdPowerOfKConsistentHashing

Threshold Power-of- k combines the strengths of the classic Power-of- k approach with a safeguard against making bad choices. The idea is that even if we look at several candidate servers, it is possible that all of them are already too full. Standard Power-of- k would still choose the best among these bad options, which reinforces hotspots. The threshold mechanism prevents this by treating an entire batch of candidates as invalid whenever all of them are above the threshold. Instead of committing to an overloaded region, the algorithm escapes the hotspot by jumping to a new, unrelated part of the ring. This makes load distribution more aggressive precisely when data becomes very imbalanced, reducing the likelihood of cascading overflows and saturated server clusters

Description: This method introduces a hybrid approach. It attempts to place objects using the “Power of k ” logic but enforces a “soft” limit defined by a threshold T . If the best of the k choices is still too heavily loaded (load $\geq T$), it falls back to a rehashing/random jump strategy to find a better spot.

Mechanism:

- **Phase 1 (Probing):** Identify k potential locations.
- **Phase 2 (Selection):** Find the server with the minimum load among the k choices (L_{min}).
- **Phase 3 (Threshold Check):**
 - **If $L_{min} < T$:** Place the object at the optimal server.
 - **If $L_{min} \geq T$:** Enter a fallback loop and perform random jumps until a server with load $< T$ is found.

Algorithm 2 Threshold Power of k Placement

```
1: Input: Object  $O$ , Threshold  $T$ , Choices  $k$ 
2:  $Choices \leftarrow$  Identify  $k$  random valid servers
3:  $BestServer \leftarrow \arg \min_{s \in Choices} (Load(s))$ 
4: if  $Load(BestServer) < T$  then
5:   Place  $O$  in  $BestServer$ 
6: else
7:   while True do
8:      $Candidate \leftarrow RandomServer()$ 
9:     if  $Load(Candidate) < T$  then
10:      Place  $O$  in  $Candidate$ 
11:      break
12:     end if
13:   end while
14: end if
```

3.2 AdaptiveKConsistentHashing

Adaptive- k is based on the observation that the system should sometimes search harder for a good server, and sometimes it should not waste effort. A fixed k cannot make this tradeoff, but an adaptive one can. When most placements succeed easily, the system reduces k to save computation because the ring is already well balanced. When placements fail—meaning all k probed servers are too full—it increases k so it can search more widely. This makes the algorithm naturally responsive to load spikes and uneven traffic patterns. An additional advantage is that k is a *global* variable, shared across all operations, which makes the method simple to maintain and implement. This method is a theoretical proposal not fully implemented in the current study.

Description: This method utilizes a dynamic, global k value. Instead of a fixed k , the system adapts the computational effort of hashing based on the state of the system. If placements are “good,” k decreases; if placements are “bad,” k increases.

Mechanism:

- Maintain global variables: k (initially 1) and *Success counter* (initially 0).
- Select k potential servers.
- Evaluate the best server among them.
- If the best server is below threshold, increment success counter; otherwise, increase k and reset counter.
- Place the object accordingly.

Algorithm 3 Adaptive- k Consistent Hashing

```
1: Global variable  $k \leftarrow 1$ 
2: Global variable success  $\leftarrow 0$ 
3: function PLACEOBJECT( $x$ )
4:   Select  $k$  candidate servers
5:    $s^* \leftarrow$  server with minimum load  $L_{\min}$  among them
6:   if  $L_{\min} < T$  then
7:     success  $\leftarrow$  success + 1
8:     if success reaches stability threshold then
9:        $k \leftarrow \max(1, k - 1)$ 
10:    end if
11:  else
12:     $k \leftarrow k + 1$ 
13:    success  $\leftarrow 0$ 
14:  end if
15:  Place  $x$  on server  $s^*$ 
16: end function
```

4 Experimental Setup

To evaluate our hypothesis that the proposed ThresholdPowerOfKConsistentHashing improves load distribution, we implemented a Python-based framework for consistent hashing experiments. The framework defines a base hash ring and three variants: a simple clockwise assignment, a fixed Random-Jump assignment, and the proposed threshold-based rehashing method. Each variant manages key placement while respecting server capacity, allowing us to compare performance under identical experimental conditions.

Datasets were created to simulate different workloads. Both uniformly distributed keys and Zipfian-distributed keys were used, the latter representing real-world patterns where a few keys are highly popular while most occur infrequently. We used 763 keys across 25 servers, each with a capacity of 30. To test scaling behavior, we also generated datasets scaled by multipliers from 0.5 to 2.0, covering light to heavy load scenarios and triggering the threshold-based rehashing in some cases.

4.1 Hash Ring Variants

The core class, `ConsistentHashRing`, initializes the server ring, tracks server loads, and provides a standard hash function. The derived variants represent different key assignment strategies:

- **Clockwise Assignment:** Assigns each key to the first server with available capacity, a simple bounded-load approach.
- **Random-Jump Assignment:** Generates a fixed number of candidate

servers via rehashing and assigns the key to the first with free capacity, reducing load clustering.

- **Threshold-Based Rehashing (Proposed):** Selects a batch of candidate servers ($k = 50$) and identifies the least loaded one. If this server exceeds a utilization threshold (80%), the algorithm triggers a fallback mechanism, performing additional random jumps until a suitable server is found or a maximum attempt limit is reached.

4.2 Datasets

Two primary dataset types were used:

- **Uniform Keys:** Sequential keys distributed evenly, representing uniform request patterns.
- **Zipfian Keys:** Keys drawn from a power-law distribution ($\alpha = 1.5$) to simulate skewed access patterns common in web workloads.

Scaled versions of these datasets were generated to simulate varying load intensities, from light to heavy, allowing us to evaluate how the algorithms perform under different pressure conditions.

4.3 Simulation Workflow

The experiments follow a consistent workflow:

1. **Server Initialization:** Instantiate the chosen hash ring variant and add all servers with fixed capacity.
2. **Key Insertion:** Insert each key using the ring’s assignment method, recording the assigned server, number of attempts, and hash computations.
3. **Metrics Collection:** Measure distribution quality (standard deviation, peak-to-average ratio, min-max spread, 95th percentile, Jain index) and computational efficiency (average insertion time, total hashes, memory overhead).
4. **Repeated Simulations:** Run all variants on each dataset type and size to collect robust results, which are visualized for comparison.

4.4 Hyperparameters

The main parameters used across all experiments are:

- Number of servers = 25
- Server capacity = 30

- Number of keys = 763
- Number of candidate servers (K) = 50
- Threshold ratio = 0.8
- Zipfian exponent $\alpha = 1.5$
- Maximum rehash attempts = 100

With this framework, any researcher can regenerate datasets, instantiate hash rings, run simulations, and compute all metrics. It also allows extensions such as new workloads, different server counts, or alternative threshold policies, ensuring experiments are fully reproducible for further studies on consistent hashing strategies.

5 Results

A complete collection of all generated plots is available in the corresponding GitHub repository and can be reproduced by executing the `main_simulation.py` script. In this results section, we focus on the most relevant observations with respect to the previously formulated hypothesis.

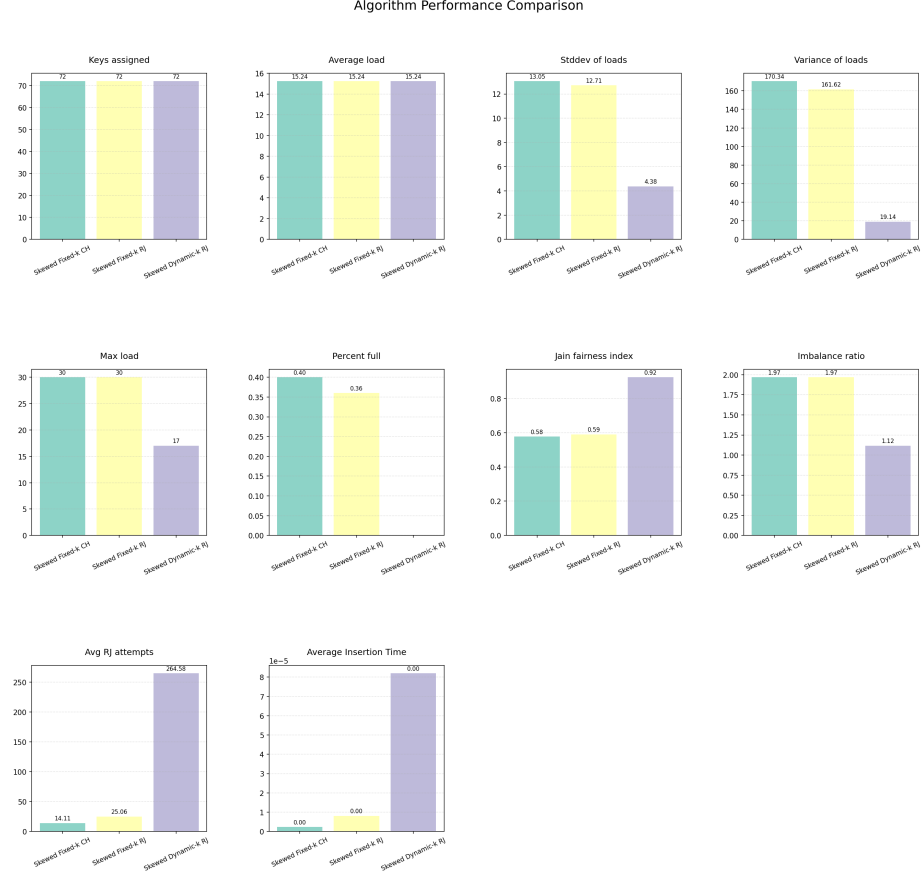


Figure 1: Skewed dataset comparison between Fixed.k_CH, Fixed.k_RJ, and Dynamic.k_RJ.

As shown in Figure 1, the proposed threshold-based approach requires a significantly higher number of average rehashing attempts before a suitable server is selected, which in turn leads to an **increased average insertion time**. This behavior is inherently dependent on the chosen threshold. Throughout all experiments, the threshold was set to 80% of the maximum server capacity. Selecting a higher threshold would likely reduce the average insertion time, as rehashing would occur less frequently.

Notably, the proportion of fully saturated servers is zero for the proposed method, accompanied by a lower average server load. In practical deployments, this behavior may result in improved system performance, provided that the increased resource utilization is an acceptable trade-off. Furthermore, both the standard deviation and variance of the server loads are consistently lower for the threshold-aware variant. This effect arises from the design of the algorithm, which actively mitigates load imbalances by rehashing until a sufficiently un-

derutilized server is identified.

We further observe that the Jain fairness index is higher for both uniform and skewed key distributions when using ThresholdPowerOfKConsistentHashing, as long as the system has not reached a state in which all servers are fully saturated, as illustrated in Figure 2. This indicates that the threshold-based approach achieves a fairer load distribution under typical operating conditions. The reduced Jain index observed in highly saturated scenarios can be attributed to newly selected servers rapidly accumulating requests, which temporarily lowers the fairness metric.

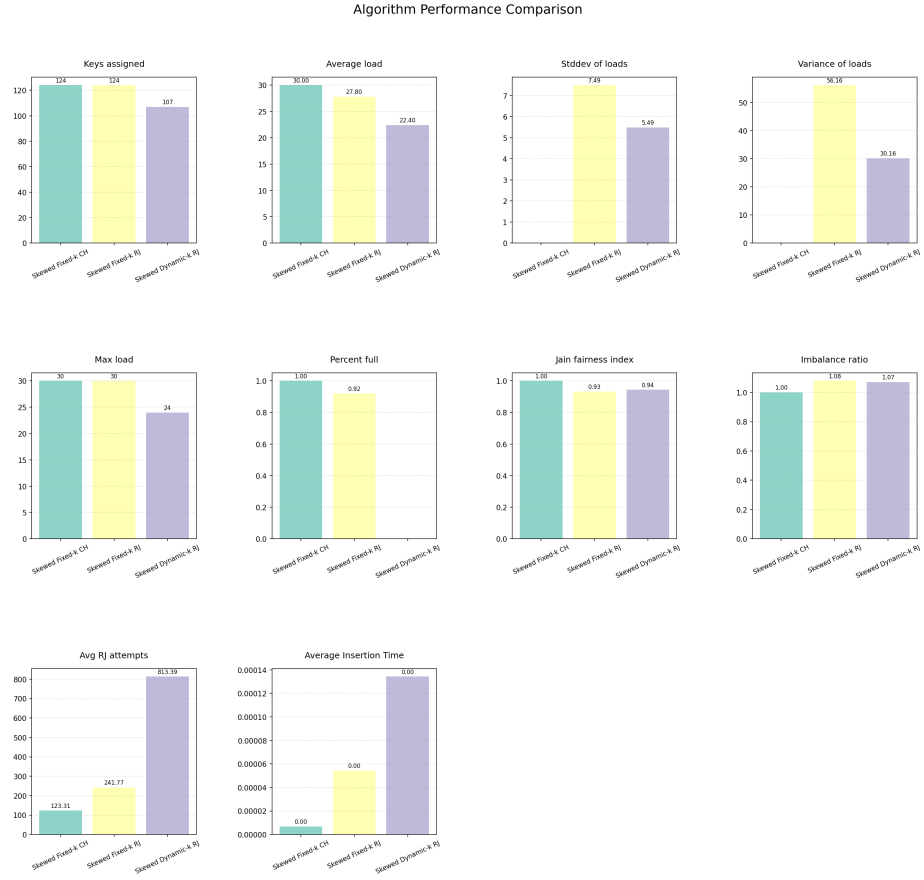


Figure 2: Skewed dataset comparison between Fixed.k-CH, Fixed.k-RJ, and Dynamic.k-RJ when all servers approach full capacity.

Overall, skewed datasets exhibit higher standard deviation and variance in server load compared to uniformly distributed keys. The threshold-aware dynamic k strategy effectively counteracts this effect by introducing controlled rehashing until a suitable server is found, thereby reducing extreme load de-

viations. In the current implementation, requests are discarded if no suitable server is identified after 100 rehashing attempts. In a production-grade system, this behavior would require further consideration, for example by introducing fallback mechanisms or adaptive thresholds.

Multiplier	Algorithm	Assigned	Avg Load	Stddev	Variance	Jain	Imbalance	Avg Attempts	Avg Insert Time (seconds)
0.50	fixed_ch	79	15.24	13.27	176.10	0.569	1.969	10.30	0.000002
0.50	fixed_rj	79	15.24	12.65	160.02	0.592	1.969	23.95	0.000009
0.50	dynamic_rj	79	15.24	4.52	20.42	0.919	1.115	241.14	0.000089
0.75	fixed_ch	90	22.88	10.74	115.31	0.819	1.311	24.11	0.000004
0.75	fixed_rj	90	22.88	11.19	125.23	0.807	1.311	51.03	0.000028
0.75	dynamic_rj	87	22.28	5.88	34.52	0.935	1.077	337.93	0.000147
1.00	fixed_ch	123	30.00	0.00	0.00	1.000	1.000	45.85	0.000003
1.00	fixed_rj	117	27.12	8.06	64.99	0.919	1.106	94.83	0.000065
1.00	dynamic_rj	102	22.68	4.74	22.46	0.958	1.058	473.75	0.000184
1.50	fixed_ch	113	30.00	0.00	0.00	1.000	1.000	130.43	0.000007
1.50	fixed_rj	117	27.84	7.37	54.37	0.934	1.078	255.09	0.000064
1.50	dynamic_rj	104	22.92	4.38	19.19	0.965	1.047	832.77	0.000147
2.00	fixed_ch	122	30.00	0.00	0.00	1.000	1.000	200.25	0.000010
2.00	fixed_rj	123	27.92	7.10	50.47	0.939	1.074	402.15	0.000069
2.00	dynamic_rj	107	23.16	4.12	16.93	0.969	1.036	1167.47	0.000155

Table 1: Comparison of load distribution metrics across algorithms for the skewed datasets. Highest values per metric are highlighted in bold.

Table 1 summarizes the evaluation results across all considered skew multipliers and hashing strategies. The table reports key load distribution metrics, including average and maximum server load, variance and standard deviation of the load distribution, as well as fairness and performance-related measures such as the Jain fairness index, imbalance ratio, average rehashing attempts, and average insertion time. For each metric, the highest observed value across all configurations is highlighted to emphasize extreme behaviors and trade-offs between the evaluated approaches.

When increasing the dataset size (going from 0.5 to 2), fixed- k variants tend to saturate servers earlier, which is reflected by higher maximum loads and, in some cases, perfect Jain index values when all servers are fully utilized. In contrast, the threshold-aware dynamic approach actively avoids server saturation by redistributing requests through repeated rehashing, resulting in lower load variance and more balanced utilization under non-saturated conditions.

While the proposed threshold-based approach consistently achieves higher Jain fairness and lower variance, this comes at the cost of increased rehashing attempts and insertion time, highlighting a clear fairness–performance trade-off.

6 Further Work

While this study demonstrated that Threshold-Based Power-of- k significantly improves load distribution metrics (such as the Jain Index) compared to fixed baselines, it introduces a computational cost in terms of higher average hash attempts. A primary avenue for future research is the implementation of the Adaptive- k algorithm proposed in Section 3.2. Developing and evaluating such an algorithm is an interesting direction, as it could further improve load balancing, reduce cascading overflows, and make the system more responsive to varying workloads.

The next step would be to implement the proposed adaptive approach, allowing k to increase or decrease depending on recent placement outcomes. More advanced extensions could explore machine learning methods to learn from past data, predicting optimal placements, adjusting k , or adapting the hash function in real-time. Such a system would aim to be self-optimizing, resilient to skewed or bursty workloads, and capable of maintaining fairness and efficiency under a wide range of conditions.

References

- [1] J. Chen, B. Coleman, and A. Shrivastava, “Revisiting consistent hashing with bounded loads,” *arXiv preprint arXiv:1908.08762*, 2020.
- [2] B. R. Coleman, A. Shrivastava, and A. Srinivasan, “Consistent hashing using the power of k choices in server placement,” Patent US 17/19 835, 2020.
- [3] V. Mirrokni, M. Thorup, and M. Zadimoghaddam, “Consistent hashing with bounded loads,” *arXiv preprint arXiv:1608.01350*, 2016.